

MetroAir Hexa + Pusher

ArduCopter Firmware Build Guide

Pusher-propeller torque feed-forward compensation

Board: MicoAir743v2 • Firmware: ArduCopter 4.6.x • Frame: Hexa-X + EDF pusher

This document is fully self-contained: it includes the source patch, parameters, and step-by-step commands to apply, build (SITL and the real **.apj**/bootloader), flash, test, and calibrate the firmware on a fresh machine. Hand it to a new chat or another engineer and they can build it end-to-end.

Airframe	262 mm hexa (131 mm adjacent), MTOW 600 g+
Lift motors	F1404 KV3800/4600, 3.5" x 2" props (6x)
Pusher	QF1611 KV6000 / 30 mm EDF on body +X
Battery	LiPo 1300 mAh 4S1P
FCU	MicoAir743v2 (ArduCopter 4.6.x)
Pusher output	SERVO7_FUNCTION = 58 (RCIN8 passthrough)

0. Why this firmware exists

Across four flight logs the airframe repeatedly pitched **nose-down on its own** when the pusher spun up, and crashed. In log 00000170.BIN the attitude controller commanded the **maximum +75°/s nose-up** recovery and **saturated all six lift motors (~1950µs)**, yet the vehicle still pitched to **-84°** and dove from 22 m. Pilot input was near neutral the whole time.

Cause: the pusher's thrust line is offset from the centre of gravity, so its thrust produces a **moment** the stock multicopter controller has no model of. It only reacts *after* an attitude error appears — too late at high pusher thrust.

Fix: feed-forward the pusher's disturbance moment so the main rotors cancel it *before* an error develops. This guide builds an ArduCopter firmware that does exactly that, gated behind parameters and off by default.

0.1 The model (torque equations)

A body +X pusher produces force and moment ($T_p, Q_p \propto \Omega^2 \propto \text{pusher}^2$):

$$\begin{array}{lcl} F_p & = & [T_p, 0, 0] & \text{forward thrust} \\ \tau_p & = & [\underbrace{\sigma Q_p}_{\substack{\text{motor} \\ \text{reaction} \\ \text{torque}}}, \underbrace{z_p T_p}_{\substack{\text{vertical} \\ \text{offset of} \\ \text{thrust } (z_p)}}, \underbrace{-h T_p}_{\substack{\text{Y-offset of} \\ \text{thrust line } (h)}}] & \text{roll, pitch, yaw} \end{array}$$

The main rotors must generate **$\tau_{\text{main}} = \tau_{\text{des}} - \tau_p$** , i.e. add **$-\tau_p$** as feed-forward. The vertical-offset **pitch** term ($z_p \cdot T_p$) is the one that crashed the aircraft; it is the primary gain to calibrate.

1. Prerequisites (one-time toolchain setup)

ArduPilot builds on macOS and Linux. Pick your OS.

1.1 macOS

```
# Xcode command-line tools + Homebrew
xcode-select --install
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Python + git
brew install python git gawk

# (for building the real board firmware) ARM cross compiler
brew install --cask gcc-arm-embedded # provides arm-none-eabi-gcc
```

1.2 Ubuntu / Debian Linux

```
sudo apt update
sudo apt install -y git python3 python3-pip gawk gcc-arm-none-eabi
```

1.3 ArduPilot Python dependencies (both OSes)

```
python3 -m pip install --user pymavlink MAVProxy empy==3.3.4 pexpect future
```

2. Get the ArduPilot source

Match the branch to the version running on the board (4.6 here). The `--recurse-submodules` flag is required.

```
git clone --recurse-submodules -b Copter-4.6 \
  https://github.com/ArduPilot/ardupilot.git
cd ardupilot
# if you forgot the submodules:
git submodule update --init --recursive
```

Also clone (or copy) the project repo that holds the patch and params:

```
git clone <your PARAGLIDER-RTG repo url>
# the firmware package lives in:
#   PARAGLIDER-RTG/firmware/copter_pusher/
```

3. Apply the pusher feed-forward patch

From inside the ardupilot checkout:

```
git apply /path/to/PARAGLIDER-RTG/firmware/copter_pusher/ardupilot_patch/pusher_ff.patch

# verify it went in:
grep -c "Pusher (X-axis propulsion) torque feed-forward" \
  libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.cpp # -> 1
```

If `git apply` reports rejects (line numbers drift between point releases), apply the four edits by hand — only two files change. The exact blocks are in Appendix A of this document.

4. Build & test in SITL first (no hardware risk)

SITL's stock hexa has no pusher, so it cannot reproduce the aerodynamic disturbance — but it **does** confirm the patch is wired correctly: with the compensation ON, sweeping the pusher RC channel injects a pitch command proportional to pusher².

```
./waf configure --board sitl
./waf copter

# launch a hexa SITL
Tools/autotest/sim_vehicle.py -v ArduCopter -f hexa --console

# in a second terminal, run the probe:
python3 /path/to/PARAGLIDER-RTG/firmware/copter_pusher/sim/sitl_pusher_probe.py \
  --conn udp:127.0.0.1:14550 --gain 0.12
```

Or run the whole thing with one wrapper:

```
/path/to/PARAGLIDER-RTG/firmware/copter_pusher/sim/run_sitl_mac.sh \
  /path/to/ardupilot /path/to/PARAGLIDER-RTG
```

PASS criteria: the EN=0 column stays ~flat as pusher PWM rises; the EN=1 column shows pitch growing with pusher (sign follows ATC_PUSH_PIT). That proves the term is live and reads the right channel before you ever fly it.

5. Build the real firmware (.apj + bootloader)

For the MicoAir743v2 flight controller:

```
./waf configure --board MicoAir743v2
./waf copter

# flashable firmware image:
#   build/MicoAir743v2/bin/arducopter.apj
#   build/MicoAir743v2/bin/arducopter      (elf, for debugging)

# bootloader (only if you must (re)flash the bootloader):
./waf configure --board MicoAir743v2 --bootloader
./waf bootloader
#   build/MicoAir743v2/bootloader/MicoAir743v2_bl.bin
```

The `.apj` is what you give your friend / load in Mission Planner. The bootloader rarely needs reflashing — only do it if instructed.

6. Flash the firmware

6.1 Mission Planner (recommended)

Setup → Install Firmware → **Load custom firmware** → pick arducopter.apj. Connect the board over USB in DFU/normal mode as prompted.

6.2 Command line (uploader)

```
python3 ardupilot/Tools/scripts/uploader.py \
  build/MicoAir743v2/bin/arducopter.apj
```

7. Load the vehicle configuration

Load params/metroair_pusher.param (Mission Planner → Config → Full Parameter List → Load from file → Write). It also moves the flight-mode switch **off RC8 to RC7** and enables the **battery failsafes** — both contributed to the crashes. The full file is in Appendix B.

8. Calibration procedure (safety critical)

The compensation term is new and untested on your airframe. Bring it up incrementally; never start with a guessed non-zero gain.

- 1 **Mechanical first.** The best fix is to put the EDF thrust line through the CG height. The feed-forward handles the *residual* offset. Until the pitch gain is calibrated, do NOT run the pusher above ~50%.
- 2 **Flight 0 — prove it's stock.** With ATC_PUSH_EN=0, fly a normal ALT_HOLD hover. Behaviour must be unchanged. (Move the mode switch to RC7 first.)
- 3 **Measure the disturbance.** Put lua/pusher_calib.lua in APM/scripts/, set SCR_ENABLE=1. Fly ALT_HOLD, keep the pitch stick neutral, step the pusher 10-20-30-40-50%. After landing read the PUSH-CAL table on the GCS: mean pitch per pusher band. Negative pitch at higher pusher = nose-down disturbance.
- 4 **Tune ATC_PUSH_PIT.** Set ATC_PUSH_EN=1, start ATC_PUSH_PIT=0.02. Repeat the stepped test. If the nose still drops, increase 0.01-0.02 at a time; if it now pitches up, you overshoot (or flip the sign). Goal: mean pitch ~0 as pusher rises with pitch stick neutral.
- 5 **Trim roll/yaw.** With pitch handled, null residual roll (ATC_PUSH_RLL) and yaw drift (ATC_PUSH_YAW) the same way.
- 6 **Sanity limit.** If any |gain| must exceed ~0.3, the mechanical offset is too large — fix the mount, do not mask it in software (the simulation shows even feed-forward saturates beyond the rotors' authority).

9. New parameter reference

Parameter	Default	Meaning
ATC_PUSH_EN	0	Master enable (0 = identical to stock)
ATC_PUSH_RC	8	RC input channel that drives the pusher
ATC_PUSH_PIT	0	Pitch FF gain — positive commands nose-up (the crash-fix term)
ATC_PUSH_RLL	0	Roll FF gain — cancels pusher motor reaction torque
ATC_PUSH_YAW	0	Yaw FF gain — cancels Y-offset yaw moment

Implementation: injected in AC_AttitudeControl_Multi::rate_controller_run() right after the rate PIDs:

```
p2 = pusher_norm^2 // ~ T_p, Q_p (command^2)
set_roll( get_roll() + ATC_PUSH_RLL * p2 ) // cancels sigma*Q_p
set_pitch( get_pitch() + ATC_PUSH_PIT * p2 ) // cancels z_p*T_p (crash term)
set_yaw( get_yaw() + ATC_PUSH_YAW * p2 ) // cancels -h*T_p
```

Appendix A — Source patch (hand-apply reference)

Two files in `libraries/AC_AttitudeControl/`. If the `.patch` rejects, make these edits manually. Indices 30-34 must be unique in the `var_info` table (renumber if taken).

A.1 `pusher_ff.patch`

```
diff --git a/libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.h b/libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.h
index 1111111..2222222 100644
--- a/libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.h
+++ b/libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.h
@@ -- Member variables (add near the other AP_Param members, e.g. after _thr_mix_man) --
    AP_Float          _thr_mix_man;          // throttle vs attitude control prioritisation used when us
+
+ // ---- Pusher (X-axis propulsion) torque feed-forward compensation ----
+ // Cancels the disturbance moments produced by a body +X pusher prop:
+ //   pitch : thrust line vertical offset from CG (the term that crashed 00000170.BIN)
+ //   roll  : pusher motor reaction torque      (sigma * Q_p)
+ //   yaw   : thrust line Y offset from CG      (-h * T_p)
+ // All gains default to 0 -> identical to stock until calibrated.
+ AP_Int8          _push_en;                // enable pusher compensation
+ AP_Int8          _push_rc;                // RC input channel driving the pusher (1..16)
+ AP_Float         _push_pit;                // pitch feed-forward gain (normalised per pusher_norm^2)
+ AP_Float         _push_rll;                // roll feed-forward gain
+ AP_Float         _push_yaw;                // yaw feed-forward gain
diff --git a/libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.cpp b/libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.cpp
index 3333333..4444444 100644
--- a/libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.cpp
+++ b/libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.cpp
@@ -- Add include near the top with the other includes --
#include "AC_AttitudeControl_Multi.h"
#include <AP_HAL/AP_HAL.h>
#include <AP_Math/AP_Math.h>
+ #include <RC_Channel/RC_Channel.h> // pusher feed-forward reads the pusher RC channel
@@ -- In AC_AttitudeControl_Multi::var_info[], insert before the final AP_GROUPEND --
    AP_GROUPINFO("THR_MIX_MAN", 6, AC_AttitudeControl_Multi, _thr_mix_man, AC_ATTITUDE_CONTROL_MAN_DEFAULT),
+
+ // @Param: PUSH_EN
+ // @DisplayName: Pusher torque compensation enable
+ // @Description: Enable feed-forward compensation of an X-axis pusher propeller's disturbance moments.
+ // @Values: 0:Disabled,1:Enabled
+ // @User: Advanced
+ AP_GROUPINFO("PUSH_EN", 30, AC_AttitudeControl_Multi, _push_en, 0),
+
+ // @Param: PUSH_RC
+ // @DisplayName: Pusher RC input channel
+ // @Description: RC input channel (1..16) that drives the pusher throttle slider.
+ // @Range: 1 16
+ // @User: Advanced
+ AP_GROUPINFO("PUSH_RC", 31, AC_AttitudeControl_Multi, _push_rc, 8),
+
+ // @Param: PUSH_PIT
+ // @DisplayName: Pusher pitch feed-forward gain
+ // @Description: Normalised pitch moment added to the main rotors per (pusher_norm^2). Positive commands
+ // @Range: -0.5 0.5
+ // @Increment: 0.005
+ // @User: Advanced
+ AP_GROUPINFO("PUSH_PIT", 32, AC_AttitudeControl_Multi, _push_pit, 0.0f),
+
+ // @Param: PUSH_RLL
+ // @DisplayName: Pusher roll feed-forward gain
+ // @Description: Normalised roll moment added per (pusher_norm^2) to cancel pusher motor reaction torque
+ // @Range: -0.5 0.5
+ // @Increment: 0.005
+ // @User: Advanced
+ AP_GROUPINFO("PUSH_RLL", 33, AC_AttitudeControl_Multi, _push_rll, 0.0f),
+
+ // @Param: PUSH_YAW
+ // @DisplayName: Pusher yaw feed-forward gain
+ // @Description: Normalised yaw moment added per (pusher_norm^2) to cancel the yaw moment from a Y-off
```

```

+
+     AP_GROUPEND
+ };
@@ -- At the END of AC_AttitudeControl_Multi::rate_controller_run(), insert just before control_monitor_update()
+     _motors.set_yaw_ff(get_rate_yaw_pid().get_ff() * _feedforward_scalar);
+
+     // =====
+     // Pusher (X-axis propulsion) torque feed-forward compensation
+     // =====
+     // A body +X pusher prop produces, per the torque model:
+     //     F_p   = [T_p, 0, 0]                (forward thrust)
+     //     tau_p = [ sigma*Q_p , z_p*T_p , -h*T_p ] (roll, pitch, yaw)
+     // where T_p, Q_p ~ Omega^2 ~ (pusher command)^2.
+     // The main rotors must generate tau_main = tau_des - tau_p, i.e. ADD
+     // -tau_p as feed-forward so the disturbance is cancelled BEFORE an
+     // attitude error develops (instead of the rate loop reacting late and
+     // saturating, as it did in crash 00000170.BIN).
+     // Gains fold in all unknown constants (z_p, h, sigma, kT, kQ) and are
+     // calibrated empirically. Sign is chosen by the user during bring-up.
+     if (_push_en) {
+         float pusher_norm = 0.0f;
+         if (rc().has_valid_input()) {
+             const RC_Channel *pch = rc().channel(constrain_int16(_push_rc, 1, 16) - 1);
+             if (pch != nullptr) {
+                 // norm_input_ignore_trim() -> -1..+1 over [min,max]; map idle(min)->0, full(max)->1
+                 pusher_norm = (pch->norm_input_ignore_trim() + 1.0f) * 0.5f;
+             }
+         }
+         pusher_norm = constrain_float(pusher_norm, 0.0f, 1.0f);
+         const float p2 = pusher_norm * pusher_norm;      // thrust/torque ~ command^2
+
+         _motors.set_roll(_motors.get_roll() + _push_rll * p2);
+         _motors.set_pitch(_motors.get_pitch() + _push_pit * p2);
+         _motors.set_yaw(_motors.get_yaw() + _push_yaw * p2);
+     }
+
+     _sysid_ang_vel_body.zero();
+     _actuator_sysid.zero();
+
+     control_monitor_update();
+ }

```

A.2 MANUAL_APPLY.md (exact edit blocks)

Manual application of the pusher feed-forward compensation

If ``git apply pusher_ff.patch`` reports rejects (line numbers drift between ArduCopter point releases), apply these three edits by hand. They only touch **two files**, both in ``libraries/AC_AttitudeControl/``.

The logic uses read-modify-write (``_motors.get_pitch()`` + `ff` -> ``set_pitch()``), so it does **not** depend on the exact rate-PID call signature of your version.

Edit 1 – ``AC_AttitudeControl_Multi.h``

Find the member variable ``_thr_mix_man`` and add the pusher members right after it:

```
```cpp
 AP_Float _thr_mix_man;

 // ---- Pusher (X-axis propulsion) torque feed-forward compensation ----
 AP_Int8 _push_en; // enable
 AP_Int8 _push_rc; // RC input channel driving the pusher (1..16)
 AP_Float _push_pit; // pitch FF gain (normalised per pusher_norm^2)
 AP_Float _push_rll; // roll FF gain
 AP_Float _push_yaw; // yaw FF gain
```
```

Edit 2 – ``AC_AttitudeControl_Multi.cpp`` (includes)

Near the top with the other ``#include`` lines, add:

```
```cpp
#include <RC_Channel/RC_Channel.h>
```
```

Edit 3 – ``AC_AttitudeControl_Multi.cpp`` (parameter table)

In ``const AP_Param::GroupInfo AC_AttitudeControl_Multi::var_info[]``, find the ``THR_MIX_MAN`` entry and the trailing ``AP_GROUPEND``. Insert these five entries **before** ``AP_GROUPEND``:

```
```cpp
// @Param: PUSH_EN
// @DisplayName: Pusher torque compensation enable
// @Values: 0:Disabled,1:Enabled
// @User: Advanced
AP_GROUPINFO("PUSH_EN", 30, AC_AttitudeControl_Multi, _push_en, 0),

// @Param: PUSH_RC
// @DisplayName: Pusher RC input channel
// @Range: 1 16
// @User: Advanced
AP_GROUPINFO("PUSH_RC", 31, AC_AttitudeControl_Multi, _push_rc, 8),

// @Param: PUSH_PIT
// @DisplayName: Pusher pitch feed-forward gain
// @Range: -0.5 0.5
// @Increment: 0.005
// @User: Advanced
AP_GROUPINFO("PUSH_PIT", 32, AC_AttitudeControl_Multi, _push_pit, 0.0f),

// @Param: PUSH_RLL
// @DisplayName: Pusher roll feed-forward gain
// @Range: -0.5 0.5
// @Increment: 0.005
// @User: Advanced
AP_GROUPINFO("PUSH_RLL", 33, AC_AttitudeControl_Multi, _push_rll, 0.0f),

// @Param: PUSH_YAW
// @DisplayName: Pusher yaw feed-forward gain
// @Range: -0.5 0.5
```

```

 // @Increment: 0.005
 // @User: Advanced
 AP_GROUPINFO("PUSH_YAW", 34, AC_AttitudeControl_Multi, _push_yaw, 0.0f),
 ...

> **Index check:** indices 30-34 must be unique within this table. Verify:
> ```bash
> grep -oE 'AP_GROUPINFO\("[A-Z_]+", [0-9]+' libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.cpp
> ```
> If 30-34 are taken, renumber to any free values (the number is just a storage
> key; the param name `ATC_PUSH_*` is unchanged).

Edit 4 – `AC_AttitudeControl_Multi.cpp` (the compensation, in `rate_controller_run()`)

At the **end** of `void AC_AttitudeControl_Multi::rate_controller_run()`, insert
this block **just before** `control_monitor_update();`:

```cpp
// ---- Pusher (X-axis propulsion) torque feed-forward compensation ----
// Main rotors generate tau_main = tau_des - tau_p, i.e. add -tau_p as
// feed-forward so the pusher disturbance is cancelled BEFORE an attitude
// error develops. T_p, Q_p ~ (pusher command)^2. Gains fold in z_p, h, sigma,
// kT, kQ and are calibrated on the bench/in flight. Default 0 = stock.
if (_push_en) {
    float pusher_norm = 0.0f;
    if (rc().has_valid_input()) {
        const RC_Channel *pch = rc().channel(constrain_int16(_push_rc, 1, 16) - 1);
        if (pch != nullptr) {
            pusher_norm = (pch->norm_input_ignore_trim() + 1.0f) * 0.5f; // idle->0, full->1
        }
    }
    pusher_norm = constrain_float(pusher_norm, 0.0f, 1.0f);
    const float p2 = pusher_norm * pusher_norm;
    _motors.set_roll(_motors.get_roll() + _push_rll * p2);
    _motors.set_pitch(_motors.get_pitch() + _push_pit * p2);
    _motors.set_yaw(_motors.get_yaw() + _push_yaw * p2);
}
```

Verify it compiled in

```bash
grep -c "Pusher (X-axis propulsion) torque feed-forward" \
    libraries/AC_AttitudeControl/AC_AttitudeControl_Multi.cpp
# expect: 1 (the rate_controller_run block; the var_info comment may add more)
```

After flashing, the params `ATC_PUSH_EN`, `ATC_PUSH_RC`, `ATC_PUSH_PIT`,
`ATC_PUSH_RLL`, `ATC_PUSH_YAW` will appear in Mission Planner.

```

## Appendix B — metroair\_pusher.param

```
=====
MetroAir Hexa + Pusher - full config for the pusher-compensated firmware
Board: MicoAir743v2 Firmware: ArduCopter 4.6.x + pusher_ff patch
Load: Mission Planner -> Config -> Full Parameter List -> Load from file
=====

---- Frame (confirm; these are correct in the logs) ----
FRAME_CLASS,2 # Hexa
FRAME_TYPE,1 # X
SERVO7_FUNCTION,58 # RCIN8 passthrough -> pusher ESC (independent forward thrust)

---- Get the flight-mode switch OFF the pusher channel ----
Plug a real 3-position mode switch into RC7. Pusher stays on RC8.
FLTMODE_CH,7
FLTMODE1,2 # RC7 low -> ALT_HOLD (use for forward pusher flight)
FLTMODE2,5 # RC7 middle -> LOITER
FLTMODE3,0 # RC7 high -> STABILIZE (manual fallback)
FLTMODE4,2
FLTMODE5,5
FLTMODE6,0

---- Failsafes (were OFF in the crash logs) ----
BATT_FS_LOW_ACT,2 # low voltage -> RTL
BATT_FS_CRIT_ACT,1 # crit voltage -> LAND
RC4_TRIM,1517 # yaw trim fixed earlier

---- Extra pitch authority (helps the loop resist the pusher moment) ----
ATC_RAT_PIT_FF,0.08
ATC_RAT_PIT_P,0.115
ATC_RAT_PIT_I,0.115

=====
PUSHER TORQUE FEED-FORWARD (new firmware feature)
Start with EVERYTHING AT ZERO. Flight must be identical to stock first.
Then calibrate per firmware/copter_pusher/README.md (Section: Calibration).
=====
ATC_PUSH_EN,0 # <- leave 0 for the first flight; set 1 only when calibrating
ATC_PUSH_RC,8 # pusher is on RC8
ATC_PUSH_PIT,0.0 # pitch FF gain (the term that stops the nose-down dive)
ATC_PUSH_RLL,0.0 # roll FF gain (cancels pusher reaction torque)
ATC_PUSH_YAW,0.0 # yaw FF gain (cancels Y-offset yaw moment)

SAFETY: until ATC_PUSH_PIT is calibrated, do NOT run the pusher above ~50%.
The mechanical fix (EDF thrust line through the CG height) is still the
primary solution; the feed-forward is a backstop for residual offset.
```

## Appendix C — pusher\_calib.lua (calibration logger)

Place in the autopilot's APM/scripts/ folder, set SCR\_ENABLE=1. Reports the pusher-induced pitch/roll/yaw per pusher band so you can choose the ATC\_PUSH\_\* gains.

```
-- pusher_calib.lua - calibration aid for ATC_PUSH_* feed-forward gains
--
-- Lua cannot inject control moments, so the COMPENSATION itself lives in C++
-- (see ../ardupilot_patch/). This script only MEASURES the disturbance so you
-- can choose the gains.
--
-- Method: while the pilot's PITCH stick is neutral (so desired pitch ~ 0), the
-- actual pitch angle IS the pusher-induced disturbance. We bin the steady-state
-- pitch (and roll, and yaw rate) by pusher level and report the averages.
--
-- * A negative mean pitch at high pusher => nose-down disturbance
-- => set ATC_PUSH_PIT POSITIVE (commands nose-up) until the mean -> ~0.
-- * Residual mean roll => trim ATC_PUSH_RLL.
-- * Residual mean yaw-rate => trim ATC_PUSH_YAW.
--
-- Put this file in the autopilot's APM/scripts/ folder, set SCR_ENABLE=1.
-- Pusher assumed on RC8 (edit PUSHER_CH below if different).

local PUSHER_CH = 8 -- RC channel that drives the pusher
local PITCH_CH = 2 -- RC channel for pilot pitch (elevator)
local STICK_DZ = 40 -- PWM deadband around 1500 = "stick neutral"
local PUSHER_MIN = 1095 -- RC8 idle pwm
local PUSHER_MAX = 1935 -- RC8 full pwm
local NBINS = 10 -- 0..100% pusher in 10 bins
local REPORT_MS = 10000 -- print table every 10 s
local LOOP_MS = 100 -- 10 Hz sampling

-- per-bin accumulators: sum_pitch, sum_roll, sum_yawrate, count
local sum_pitch = {}
local sum_roll = {}
local sum_yawr = {}
local cnt = {}
for i = 1, NBINS do sum_pitch[i]=0; sum_roll[i]=0; sum_yawr[i]=0; cnt[i]=0 end

local last_report = 0
local was_armed = false

local function bin_of(pwm)
 local n = (pwm - PUSHER_MIN) / (PUSHER_MAX - PUSHER_MIN) -- 0..1
 if n < 0 then n = 0 end
 if n > 1 then n = 1 end
 local b = math.floor(n * NBINS) + 1
 if b > NBINS then b = NBINS end
 return b, n
end

local function report()
 gcs:send_text(6, "PUSH-CAL bin pusher% meanPitch meanRoll meanYawRate(deg/s) n")
 for i = 1, NBINS do
 if cnt[i] > 0 then
 local p = sum_pitch[i]/cnt[i]
 local r = sum_roll[i]/cnt[i]
 local y = sum_yawr[i]/cnt[i]
 gcs:send_text(6, string.format("PUSH-CAL %2d %3d-%3d %7.2f %7.2f %7.2f %d",
 i, (i-1)*100/NBINS, i*100/NBINS, p, r, y, cnt[i]))
 end
 end
end

function update()
 local armed = arming:is_armed()

 -- on disarm edge, dump the table once
 if was_armed and not armed then
 gcs:send_text(6, "PUSH-CAL === flight summary ===")
 report()
 end
 was_armed = armed

 if armed then
```

```

local pusher_pwm = rc:get_pwm(PUSHER_CH) or PUSHER_MIN
local pitch_pwm = rc:get_pwm(PITCH_CH) or 1500
-- only sample when pilot pitch stick is neutral (desired pitch ~ 0)
if math.abs(pitch_pwm - 1500) <= STICK_DZ then
 local b = bin_of(pusher_pwm)
 local pitch_deg = math.deg(ahrs:get_pitch())
 local roll_deg = math.deg(ahrs:get_roll())
 local gyro = ahrs:get_gyro() -- rad/s, body
 local yawrate_deg = gyro and math.deg(gyro:z()) or 0.0
 sum_pitch[b] = sum_pitch[b] + pitch_deg
 sum_roll[b] = sum_roll[b] + roll_deg
 sum_yawr[b] = sum_yawr[b] + yawrate_deg
 cnt[b] = cnt[b] + 1
end

local now = millis():toint()
if now - last_report >= REPORT_MS then
 last_report = now
 report()
end

return update, LOOP_MS
end

gcs:send_text(6, "pusher_calib.lua loaded: keep PITCH stick neutral; read PUSH-CAL table to tune ATC_PUSH_*")
return update, 1000

```

## Appendix D — Troubleshooting

| Symptom                                   | Action                                                                                                                                          |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| git apply fails                           | Use Appendix A.2 manual edits (2 files, 4 edits). Confirm you are on Copter-4.6.                                                                |
| ATC_PUSH_* not visible after flash        | The patch didn't compile in. Re-check the grep in Section 3, rebuild, reflash. Refresh params in Mission Planner.                               |
| waf configure: board not found            | Update the source (newer ArduPilot) or check exact board name: <code>./waf list_boards   grep -i micoair</code>                                 |
| submodule errors during build             | <code>git submodule update --init --recursive</code>                                                                                            |
| arm-none-eabi-gcc not found               | Install the ARM toolchain (Section 1). SITL build does not need it; the board build does.                                                       |
| SITL probe shows no EN=1 effect           | Confirm <code>ATC_PUSH_EN=1</code> , <code>ATC_PUSH_RC=8</code> , <code>ATC_PUSH_PIT</code> non-zero, and that you rebuilt after changing them. |
| Nose still drops in flight at high pusher | Disturbance exceeds pitch authority: reduce pusher, then fix the EDF thrust-line height (mechanical) by moving the motor.                       |

### Build artifacts summary

```
SITL binary : build/sitl/bin/arducopter
Board firmware: build/MicoAir743v2/bin/arducopter.apj <-- flash this
Bootloader : build/MicoAir743v2/bootloader/MicoAir743v2_bl.bin
```